

Session 3 : Création de Shaders dans Babylon.js (Partie 1)

💡 Objectifs de la session

Comprendre ce qu'est un shader, pourquoi le GPU est si rapide, et écrire ses premiers shaders GLSL dans Babylon.js.

De la copie manuelle à la presse d'imprimerie

Analogie : Gutenberg et le Shader

Avant Gutenberg, chaque livre était copié à la main, lettre par lettre. C'est comme dessiner un cercle, puis un rectangle, puis une ligne : on donne des instructions une par une, exécutées en série.

Le shader est l'équivalent de la presse d'imprimerie : au lieu de dessiner chaque élément individuellement, on écrit **une fois** un programme (la matrice de caractères), et il s'applique partout simultanément.

Qu'est-ce qu'un Fragment Shader ?

Définition

Un shader est un programme exécuté sur la carte graphique (GPU). Contrairement aux API de dessin classiques où l'on dessine cercle, rectangle, triangle un par un, un shader exécute **les mêmes instructions sur chaque pixel de l'écran**.

Le programme est une fonction à laquelle on passe une position et qui renvoie une couleur :

$$\text{couleur} = f(\text{position})$$

Une fois compilé, ce programme s'exécute extrêmement rapidement.

Pourquoi les shaders sont-ils rapides ?

CPU : un tuyau, GPU : une grille de tuyaux

Imaginez le CPU comme un gros tuyau : les opérations passent une par une. Les ordinateurs récents ont plusieurs tuyaux (threads), mais ils restent limités.

Le GPU (Graphics Processing Unit) est une **grille** de milliers de petits microprocesseurs qui tournent en parallèle. Chaque pixel est une petite opération ; au lieu de les traiter séquentiellement, le GPU les traite tous simultanément.

💬 Le calcul en parallèle

Sur un écran 800x600 à 60 FPS : $480\,000 \text{ pixels} \times 60 = \mathbf{28\,800\,000 \text{ calculs par seconde}}$.

Sur un écran Rétina 2880x1800 à 60 FPS : $\mathbf{311\,040\,000 \text{ calculs par seconde}}$.

Un CPU seul ne peut pas suivre. Le GPU résout ce problème par le traitement parallèle : des milliers de petits cœurs traitent les pixels simultanément.

💡 Bonus : accélération matérielle

Le GPU accélère aussi certaines fonctions mathématiques directement au niveau du matériel : transformations de matrices, fonctions trigonométriques, etc. Ces opérations sont traitées à la vitesse de l'électricité.

Pourquoi les shaders font peur ?

Les deux contraintes des threads GPU

Pour fonctionner en parallèle, chaque thread doit être **indépendant** des autres. Cela implique deux contraintes :

- **Cécité** : un thread ne peut pas voir ce que font les autres threads. Il est impossible de vérifier le résultat d'un autre thread ou de transmettre des données entre threads.
- **Amnésie** : dès qu'un thread a fini son traitement, le GPU lui ré-assigne une autre opération. Le thread ne garde **aucune mémoire** de ce qu'il faisait la frame précédente. Il pourrait dessiner un bouton, puis un bout de ciel, puis du texte.

💡 Conséquence pour le code

Coder un shader demande un niveau d'abstraction différent : il faut écrire une fonction **générique** qui sait rendre une image entière uniquement à partir de la variation d'une position, sans aucun état persistant.

Le Pipeline de Rendu Programmable

Étapes du pipeline

1. **Vertex Shader** : Transforme chaque sommet de l'Object Space au Clip Space.
2. **Rasterization** : Le GPU convertit les triangles en pixels.
3. **Fragment Shader** : Calcule la couleur de chaque pixel (la fonction $f(\text{position}) \rightarrow \text{couleur}$).
4. **Output Merger** : La couleur finale est écrite dans le framebuffer.

GLSL : Le langage des Shaders

Qu'est-ce que le GLSL ?

GLSL (OpenGL Shading Language) est le langage pour écrire des shaders OpenGL. Les spécifications sont maintenues par le **Khronos Group**. Il existe d'autres langages selon les plateformes (HLSL pour DirectX, MSL pour Metal), mais nous nous concentrerons sur GLSL.

Syntaxe GLSL de base

```
// Vertex Shader : transforme la position du sommet
attribute vec3 position;
uniform mat4 worldViewProjection;

void main() {
    gl_Position = worldViewProjection * vec4(position, 1.0);
}
```

```
// Fragment Shader : la fonction f(position) -> couleur
// Ici, tous les pixels seront rouges
void main() {
    gl_FragColor = vec4(1.0, 0.0, 0.0, 1.0); // Rouge
}
```

Décryptage du code

- `gl_FragColor` est une variable **globale** fournie par GLSL. Elle représente la couleur de sortie du pixel. Le GPU lit cette variable à la fin de `main()` et l'écrit dans le framebuffer.
- `vec4(1.0, 0.0, 0.0, 1.0)` est un vecteur à 4 composantes : (R, G, B, A) . Les valeurs vont de 0.0 à 1.0 (pas de 0 à 255).
 - $R = 1.0$: rouge au maximum.
 - $G = 0.0, B = 0.0$: pas de vert ni de bleu.
 - $A = 1.0$: opacité complète (pixel opaque).
- **Résultat** : tous les pixels de la géométrie sont colorés en rouge pur.

💡 Rappelez-vous

Le fragment shader s'exécute sur **chaque pixel** simultanément. Chaque instance est aveugle et amnésique : elle ne connaît que sa propre position. `gl_FragColor` est le seul moyen de communiquer le résultat au GPU.

Shaders dans Babylon.js : ShaderMaterial

Créer un *ShaderMaterial*.

```
BABYLON.Effect.ShadersStore["customVertexShader"] = `
    attribute vec3 position;
    uniform mat4 worldViewProjection;
    varying vec3 vPosition;

    void main() {
        vPosition = position;
        gl_Position = worldViewProjection * vec4(position, 1.0);
    }
`;

BABYLON.Effect.ShadersStore["customFragmentShader"] = `
    varying vec3 vPosition;

    void main() {
        gl_FragColor = vec4(vPosition.x, vPosition.y, vPosition.z, 1.0);
    }
`;

const shaderMaterial = new BABYLON.ShaderMaterial(
    "shader", scene, { vertex: "custom", fragment: "custom" },
    { attributes: ["position"], uniforms: ["worldViewProjection"] }
);
mesh.material = shaderMaterial;
```

Uniforms et Varyings

Communication entre shaders

- **Uniforms** : Variables passées du CPU (JavaScript) au GPU (shader). Constantes pour tous les sommets/pixels.
- **Attributes** : Données par sommet (position, normal, UV).
- **Varyings** : Variables passées du Vertex Shader au Fragment Shader (interpolées par pixel).